

Detyra 2

NEVER TRUST THE CLIENT.

Organizimi/arkitektura e aplikacionit dhe mbrojtja e te dhenave

Per mbrojtjen dhe aksesimin e te dhenave, propozoj nje backend te organizuar sipas arkitektures controller, service, repository, entity, DTO.

Modulet teknike te propozuara jane keto:

- **controller**: Merr kerkesat HTTP (per konsistence te perdoret nje standard emerimi 'api/v1/users'). Te implementohet autorizimi dhe rate limiting.
- **service**: Ne momentin qe kerkesa HTTP pranohet, service proceson logjiken e punes dhe pergatit pergjigjen qe do ti jepet klientit.
- **repository**: Perdoret per aksesim te database ne lexim dhe/ose ne shkrim.
- **entity**: Pershkruan strukturen e te dhenave ne database. Mund te jete nje objekt (per shembull objekt Java/Kotlin) i serializueshem.
- **DTO**: Si entity po me informacione te limituara. Ne momentin qe entity merret nga database, transformohet ne DTO. Perdoret per te limituar ekspozimin e te dhenave sensitive, p.sh: Entity 'User' permban atributin created_at, updated_at, password_hash, informacione qe nuk ndikojne funksionimin e frontend.
- **auth**: Login, leshimi dhe rifreskimi i tokenit JWT (stateless) dhe revokimi pas logout. Per passwordet, perdorimi i sherbimeve te treta (edhe open-source) qe implementojne praktikat me te mira hashing dhe storing per te evituar vulnerabilitete autentikimi.
- **users**: Mbledhje minimale e te dhenave gjate regjistrimit. Percaktimi i roleve me autorizime te ndryshme, ku roli default eshte roli me privilegjet me te ulta, ndersa roli i admin leshohet vetem nga admin.
- **catalog**: Implementimi i veprimeve CRUD. Perdorimi i ORM per te evituar probleme (apo sulme) me query. Perdorimi i DTO per te limituar ekspozimin panevojshem e disa te dhenave. **KUJDES**: Perdorimi i ORM eshte efikas, por vjen me probleme qe duhen evituar sic eshte problemi 1+N.
- **orders**: Implementimi i veprimeve CRUD me kujdes te vecante ne transferimin e gjendjes se porosis (nga pending ne completed).
- **notifications**: Kontroll periodik i stock.
- **exceptions**: Exception te personalizuar per gabime qe mund te ndodhin. Frontend supozohet ta pengoje perdoruesin per te bere kerkesa jo te vlefshme, sic eshte porosia e produkteve jasht stocku. Backend duhet te jete gjithesesi gati ta menaxhoje nje rast te tille dhe te kape gabimin (exception) te personalizuar (OutOfStockException)

Rolet dhe privilegjet

Roli	Veprimet
Operator	Shfletim, kerkim produktesh + krijim porosishe.
Merchant	Veprime CRUD produktesh, operatoresh dhe pamje statistikash

Autorizimi kryhet permes JWT ku backend i kupton vete privilegjet e perdoruesit qe ben kerkesen, permes dekodifikimit dhe verifikimit te tokenit.

Entitetet e database

```
User          id, email, password_hash, role (OPERATOR | MERCHANT),
created_at, updated_at
Merchant       id, user_id (FK → User), company_name, verified (bool)
Product       id, merchant_id (FK → Merchant), name, category, price,
stock, expires_at
Order         id, operator_id (FK → User), status (PENDING | CONFIRMED |
CANCELLED), created_at
OrderItem     id, order_id (FK → Order), product_id (FK → Product),
quantity, unit_price
```

Notes:

- `unit_price` eshte pjese e `OrderItem` pasi nese cmimi ndryshon, historiku i porosive nuk ndryshon. Nese do te perdornim `price` te entitetit `Product` , nje ndryshim ne cmim do te ndryshonte totalin e porosive te shkuara.
- `User` mund te permbante informacionet e entitetit `Merchant` , por preferoj ndarjen e entitetve per te mos mbipopulluar database me vlera NULL.

API endpoints

```
POST    /auth/login
POST    /auth/register

GET     /products          list/search/filter (te gjitha rolet)
GET     /products/{id}
POST    /products          Merchant – krijimi
PUT     /products/{id}     Merchant – modifikim
DELETE  /products/{id}     Merchant – fshirje

POST    /orders            Operator – krijim
GET     /orders            Operator - historiku i porosive
Merchant: porosite e bera nga Operator
GET     /orders/{id}
```

PATCH /orders/{id}/status Merchant: modifikim statusi · Operator: anulim porositë (vetëm nëse statusi është pending)

(GET /products, GET /orders) duhet të kenë pagination dhe parametra filtrimi.

MVP dhe faza 2

MVP

- Auth (login/register), me role.
- Merchant CRUD.
- Operator browse/search/filter + porositë.
- Pranimi ose refuzimi i porositë nga Merchant.
- Historiku i porositë.

Faza 2

- Notifications (stock, skadenca, statusi i porositë (kur ka kohë që nuk janë pranuar ose refuzuar).
- Statistik dhe reportim
- Multi-warehouse dhe stock në baze të vendndodhjes.
- Integrim pagesash.
- Rol SUPER-ADMIN për mbikqyrje të të gjithë përdoruesve.
- (Teknike, nëse rriten përmasat) Transicion nga arkitektura monolitike në arkitektura micro-services.